

iTarang Ops Console — Development Plan

Monitoring dashboard for the tech team. Route `/operations` · role `operations` · single login `operations@itarang.com` .

Status: **planned, not started**. Owner: TBD. Estimated ~12.5 developer-days across 8 phases.

0. Why

There is no single place to see whether iTarang is healthy. Today the answer is scattered across `/api/health` (curl only), pm2 on two VPS boxes (SSH only), AWS RDS (`pg_stat_*` via psql only), ten vendor billing portals, and `/Leads → Cost Analytics` for AI call spend. A full disk, a dead background ticker, or a burnt ElevenLabs credit balance is currently invisible until a user reports a broken page.

This document is the development plan for a dashboard that puts all of it on one screen.

Agreed constraints:

- Lives **inside the existing CRM** as a new route group + role. No new app, no new infra, no new hosting, no new deploy pipeline.
- Server metrics and logs arrive via a **lightweight push agent** on each box. The CRM never SSHes into anything.
- A daily job writes **DB snapshots**, drives a **one-slide summary page**, and raises **threshold alerts**. (Emailed daily digest is backlog, not v1.)
- Build is **phased** — foundation → infrastructure → spend → business. Every phase ships.

1. What we are building

A self-contained section of the CRM that answers, on one screen:

- Are the servers alive and do they have headroom?
- Is the AWS RDS database healthy?
- What is erroring right now, and how much?
- What are we spending on third-party APIs, and are any credits about to run out?
- Are the business numbers moving?
- Is the monitoring itself still running?

Design principle 1 — the monitor is a thin layer. Almost every number already exists somewhere in this codebase. The job is to collect it on a schedule, freeze a daily snapshot, and render it. Do not re-write SQL that `/api/dashboard/ceo/overview` already owns.

Design principle 2 — adding a new thing to monitor must be a one-file change. Everything routes through a metric registry plus a collector interface (§4).

Design principle 3 — pages never call vendors. A page render reads tables only. If a vendor API is slow or down, the dashboard still loads and shows stale-with-timestamp.

2. Where it plugs into the existing codebase

Read this table before writing any code. Most of the build is assembly, not invention.

Concern	Existing thing to reuse	Path
Route shell, sidebar, header	<code>(dashboard)</code> layout — free for any new route group	<code>src/app/(dashboard)/layout.tsx</code>
Reference implementation — a single-role dashboard, done well	The IT/Security console. Copy its shape wholesale.	<code>src/app/(dashboard)/it/</code>
API auth guard	<code>requireSecurityAdmin()</code> — copy verbatim, swap the role set	<code>src/lib/security/route-guard.ts</code>
Page auth guard	<code>requireAuth()</code> + an <code>ALLOWED_ROLES</code> set	<code>src/lib/auth-utils.ts</code>
Job lifecycle (lock, reap, freshness)	<code>beginScan</code> / <code>completeScan</code> / <code>failScan</code> / <code>reapStaleRuns</code> / <code>getScanFreshness</code>	<code>src/lib/security/scan-run.ts</code>
The only scheduler that actually runs in prod	in-process tickers	<code>src/instrumentation-node.ts</code> , registered in <code>src/instrumentation.ts</code>
Dependency health JSON	<code>/api/health</code> — CRM DB, IoT bridge, risk sandbox, commit SHA	<code>src/app/api/health/route.ts</code>
Redis / queue circuit state	<code>/api/health/redis</code>	<code>src/app/api/health/redis/route.ts</code>
AI call cost (ElevenLabs + Bolna)	full cost-analytics SQL, already correct and reconciled	<code>src/lib/campaigns/cost-analytics-query.ts</code>
Billed vendor spend	<code>expense_submissions</code> where <code>bucket='tech'</code>	<code>src/lib/expenses/bucketRules.ts</code>
Business metric time windows	<code>resolveWindowParams</code> / <code>resolveWindow</code>	<code>src/lib/dashboard/salesWindow.ts</code>
Business headline numbers	CEO overview aggregation	<code>src/app/api/dashboard/ceo/overview/route.ts</code>
Charts (code-split wrapper — never import recharts directly)	<code>MetricsChart</code>	<code>src/components/shared/charts.tsx</code>
UI primitives	<code>StatCard</code> , <code>Badge</code> , <code>Tabs</code> , <code>Card</code> , <code>Skeleton</code>	<code>src/components/ui/</code>
Tables	<code>DataTable</code> , <code>usePagination</code> , <code>useTableSort</code>	<code>src/components/shared/</code>
Live refresh without websockets	<code>AutoRefresh</code> — 15s <code>router.refresh()</code> , pauses on hidden tab	<code>src/app/(dashboard)/it/security/_components/AutoRefresh.tsx</code>
Final export	Final CSV exports template	<code>src/app/api/it/security/_components/Export.tsx</code>

Excel export	exceljs route template	src/app/api/it/security/report/export.xlsx/route.ts
Email (alerts; later, digest)	getMailer() — AgentMail or SMTP	src/lib/email/mailer.ts
In-app notification	notifyRoles()	src/lib/notifications/notify.ts
Rate-limited logger (built after a 60 GB log incident)	—	src/lib/log.ts
Schema drift check	npm run db:drift	scripts/db-drift.ts
Seeding a single-login role user	scripts/create-it-user.ts + npm run seed:it-user	—

Dependencies: add nothing. Everything needed is already installed — `recharts`, `@tanstack/react-query`, `exceljs`, `zod`, `drizzle-orm`, `postgres`, `ioredis`, `resend`/`nodemailer`, `@aws-sdk/client-s3`. The host agent is dependency-free Node.

3. Architecture



- Three layers, strictly separated:
- Collectors** — pure async functions. They perform no writes; they return `CollectedSample[]`.
 - Runner + storage** — one ticker owns scheduling, locking, timeouts, persistence and alert evaluation.
 - Pages** — read from the tables only.

4. The two interfaces the whole system hangs off

4.1 Metric registry — `src/lib/operations/registry.ts`

One entry per monitored number. **This is the file a teammate edits to add a metric.**

```
export type OpsModule =
  | "infra" | "database" | "logs" | "spend" | "business" | "team" | "system";

export type MetricUnit =
  | "count" | "percent" | "bytes" | "ms" | "inr_paise" | "credits" | "bool" | "days";

export interface MetricDef {
  key: string; // "host.prod.disk_used_pct" — stable, dot-namespaced
  label: string; // "Disk used — prod"
  module: OpsModule;
  unit: MetricUnit;
  direction: "higher_is_better" | "lower_is_better" | "neutral";
  warn?: number; // seed values for ops_alert_rules
  crit?: number;
  onSlide?: boolean; // show on the one-slide board
  help?: string; // tooltip: what this means, what to do when it goes red
}

export const METRICS: MetricDef[] = [ /* ... */ ];
export function getMetric(key: string): MetricDef | undefined;
```

4.2 Collector — src/lib/operations/collectors/types.ts

```
export interface CollectedSample {
  metric_key: string;
  source: string; // "rds:crm" | "host:prod" | "vendor:elevenlabs"
  value_num?: number | null;
  value_text?: string | null;
  meta?: Record<string, unknown>;
  captured_at?: Date; // defaults to now()
}

export interface OpsCollector {
  id: string; // "db.rds.crm"
  label: string;
  intervalMs: number; // how often this collector is due
  timeoutMs?: number; // default 10_000
  run(): Promise<CollectedSample[]>;
}
```

Each collector is its own file under src/lib/operations/collectors/, exported from index.ts as a flat array.

Adding a monitor = one new collector file + one or more registry entries + one line in collectors/index.ts. Nothing else in the system changes.

5. Database — migration E-227_ops_monitoring.sql

Follow the house rules in CLAUDE.md: hand-written, **idempotent** (CREATE TABLE IF NOT EXISTS , CREATE INDEX IF NOT EXISTS), strictly **additive**, mirrored into src/lib/db/schema.ts, and **add a row to drizzle/MIGRATION_CHECKLIST.md**. Apply by hand via pgAdmin to database-1 (sandbox) and database-2 (prod). Never db:push.

Six tables:

Table	Purpose	Shape / notes
ops_metric_samples	generic time series — every collector writes here	(metric_key, source, captured_at, value_num, value_text, meta jsonb). Indexes on (metric_key, captured_at DESC) and (captured_at DESC). Pruned to 30 days.
ops_daily_snapshots	one frozen row per day per metric	(snapshot_date, metric_key, source, value_num, value_min, value_max, value_avg, meta) with UNIQUE(snapshot_date, metric_key, source). Never pruned — this is the trend history and the one-slide's source.
ops_collector_runs	did the collector run, how long, did it fail	Copy security_scan_runs: collector_id, triggered_by, status, started_at, finished_at, duration_ms, samples_written, error. Partial unique index (collector_id) WHERE status='running' = single-flight lock.
ops_log_events	log lines forwarded by the agent	(host, service, level, message, logged_at, fingerprint, meta). Indexes on (logged_at DESC), (level, logged_at DESC), (fingerprint). Pruned to 14 days.
ops_alert_rules	editable thresholds	(metric_key, source, comparator, warn_threshold, crit_threshold, enabled, cooldown_minutes, notify_channels jsonb) with UNIQUE(metric_key, source). Seeded from METRICS defaults on first run.
ops_alerts	open / resolved breaches	(rule_id, metric_key, source, severity, value_num,

ops_alerts	open / resolved breaches	(rule_id, metric_key, source, severity, value_num, threshold, message, opened_at, resolved_at, notified_at, status) . Partial unique (metric_key, source) WHERE resolved_at IS NULL so a flapping metric cannot spawn duplicate rows — the same trick telemetry_alerts uses.
------------	--------------------------	---

Conventions: `value_num` is `numeric` throughout; money is **INR paise** (matching `ai_call_logs.*_cost_cents`); bytes are bytes; percents are 0–100.

6. Scheduling

Register **one** ticker, following the exact pattern already in `instrumentation-node.ts` (`inFlight` guard, staggered `setTimeout` kickoff, `interval.unref()` , early return on `VERCEL === "1"` , opt-out on `ENABLE_OPS_MONITOR === "0"`):

```
// src/instrumentation-node.ts
export async function startOpsMonitorTicker() { /* tick every 60s */ }
```

...then add `await startOpsMonitorTicker();` to `src/instrumentation.ts` inside the existing `NEXT_RUNTIME === "nodejs"` block.

⚠ **Keep it a wrapping if , never an early return.** The comment at the top of `src/instrumentation.ts` explains why: webpack only prunes statically-false `if` branches, so an early `return` leaves the whole Drizzle/postgres-js/googleapis graph in the Edge bundle and the build fails with `Can't resolve 'http'` .

The ticker calls `runDueCollectors()` every 60 s. Each collector declares its own `intervalMs` ; the runner executes only the ones whose last completed run is older than that. One timer, per-collector cadence, and `/operations/jobs` observability for free.

At **00:15 IST** the runner additionally executes `runDailySnapshot()` — freeze yesterday into `ops_daily_snapshots` , prune old samples and logs, re-evaluate alerts.

Do not use Vercel cron or BullMQ. `vercel.json` crons do not fire on the PM2 boxes, and no BullMQ worker process is declared in production (`callQueue.add()` is never called anywhere in the codebase). The in-process ticker is the only mechanism that provably runs on both sandbox and prod.

7. The host agent

`ops-agent/agent.js` — a single dependency-free Node script (~150 lines) committed to this repo, copied to each box, run by pm2 (process name `ops-agent`) or a 5-minute cron.

Reads:

- `node:os` → load average, total/free memory, uptime, CPU count
- `df -P /` and `df -Pi /` → disk and inode used %
- `free -m` → swap
- `pm2 jlist` → per-process status, restart count, memory, CPU, uptime
- tail of `logs/web.out.log` , `logs/web.err.log` and `/var/log/nginx/error.log` , tracking a byte offset in a local state file so each line is sent exactly once
- `openssl x509 -enddate` against the served cert → days to expiry

POSTs one JSON body to `POST /api/operations/ingest/host` :

```
{
  "host": "prod",
  "captured_at": "2026-08-03T12:00:00Z",
  "metrics": {
    "cpu_pct": 31.2, "load1": 0.9, "mem_used_pct": 68.1, "swap_used_pct": 0,
    "disk_used_pct": 74.3, "disk_free_gb": 22.1, "inode_used_pct": 12,
    "uptime_s": 918273, "cert_days_left": 61 },
  "processes": [{
    "name": "itarang-crm-web", "status": "online", "restarts": 3,
    "mem_mb": 612, "cpu_pct": 4.1, "uptime_s": 88123 }],
  "logs": [{
    "ts": "...", "service": "itarang-crm-web", "level": "error", "message": "..." }]}
}
```

Ingest route hardening — `src/app/api/operations/ingest/host/route.ts`

Middleware returns early for `/api/*` (see `src/middleware.ts`), so **this route must defend itself**:

- Authorization:** `Bearer <OPS_INGEST_SECRET>` , compared with `crypto.timingSafeEqual`
- `host` must appear in the `OPS_INGEST_HOSTS` allowlist
- zod-validated body, **512 KB cap**, max **200 log lines** per POST
- reject a `captured_at` skewed more than 10 minutes from server time
- per-host rate limit (1 accepted POST per minute)
- log messages truncated to 2000 chars and **fingerprinted** (hash of service + level + normalised message) so the UI groups repeats instead of drowning in them

Env: `OPS_INGEST_SECRET` , `OPS_INGEST_HOSTS=prod,sandbox,iot` .

⚠ Prod's `shared/.env` is **rewritten from the `PROD_ENV_FILE_B64` GitHub secret on every deploy** — the new vars must be added to the box *and* the GitHub secret, or they vanish on the next deploy. Sandbox's `shared/.env` is seeded once and edited on the box (changing the GitHub env secret does not propagate there).

8. Modules

8.1 One-slide dashboard — /operations

Traffic-light grid of every metric flagged `onSlide`, read from `ops_daily_snapshots` (yesterday) with live values overlaid from the newest `ops_metric_samples`. Green/amber/red comes from `ops_alert_rules`. Plus: open-alert count, deploy SHA per environment, and a "collectors last ran N minutes ago" freshness footer.

Designed to be screenshotted for standup — one screen, no scrolling, no interaction needed.

8.2 Infrastructure — /operations/infrastructure

Per host: CPU, load, memory, swap, disk %, inode %, uptime, SSL days-left, plus a pm2 process table (status, restart count, memory against the `900M max_memory_restart` ceiling in `ecosystem.prod.config.js`). 24 h sparkline per metric. Source: agent push.

8.3 Database health — /operations/database

Pulled by plain SQL against RDS itself. **No CloudWatch, no new AWS SDK client, no new IAM permissions.**

Metric	Query
connections used vs <code>max_connections</code>	<code>pg_stat_activity</code> + <code>SHOW max_connections</code>
longest running query; longest idle-in-transaction	<code>pg_stat_activity</code>
cache hit ratio, commits vs rollbacks, deadlocks	<code>pg_stat_database</code>
DB size; top-20 tables by size and by row count	<code>pg_stat_user_tables</code> , <code>pg_total_relation_size</code>
dead tuples / autovacuum age	<code>pg_stat_user_tables</code>
unapplied-migration drift	reuse <code>scripts/db-drift.ts</code>

⚠ **Highest-value monitor in the whole build.** `src/lib/db/index.ts` sets `max: 5` with the comment "RDS has ~79 `max_connections` and NO pooler; a deploy burst trips 53300". Connection headroom belongs on the one-slide board.

Run against both `db` (CRM RDS) and `getIotDb()` (IoT bridge).

Note: `src/app/api/system/database-monitor/route.ts` looks like it already does this, but it calls `getIotSql()` — it reports the **IoT VPS database, not RDS**, despite its name and its `requireRole(['ceo'])` gate. Do not reuse it as-is.

8.4 Logs & errors — /operations/logs

Explorer over `ops_log_events`: filter by host / service / level / time, text search on message, a grouped-by-fingerprint "top errors (24 h)" table with counts, and an error-rate sparkline. Click a group → recent raw lines. Excel export via the ExcelJS route template.

8.5 System usage — /operations/system

The application's own vital signs:

- **/api/health history** — per-dependency up/down and latency over time, uptime %
- **/api/health/redis** — Upstash quota circuit (`quotaCircuit.open`, `reopenAt`) and the `ENABLE_CALL_WORKER` flag
- **Job liveness** — for each background job: last run, age, status, error. Reads `zoho_sync_state`, `security_scan_runs`, `drive_scan_runs`, `risk_runs`, `scraper_runs` and `ops_collector_runs`. *This is the module that would have caught the Zoho sync ticker being dead on prod.*
- **Deploy tracking** — commit SHA per environment, and whether sandbox and prod are on different builds

8.6 Vendor usage & spend — /operations/spend

Two independent views that should reconcile against each other.

Metered usage (what we consumed):

Vendor	Source
ElevenLabs	<code>ai_call_logs</code> WHERE <code>provider='elevenlabs'</code> — calls, minutes, cost, via <code>src/lib/campaigns/cost-analytics-query.ts</code> . New work: poll <code>GET /v1/user/subscription</code> with the <code>xi-api-key</code> header for credit balance / quota / reset date. Nothing polls this today, so a credit burn-out is currently invisible.
Bolna	<code>ai_call_logs</code> WHERE <code>provider='bolna'</code>
OpenAI	<code>security_scan_runs</code> , <code>security_findings</code> , <code>risk_runs</code> , <code>risk_card_runs</code> — <code>prompt_tokens</code> / <code>completion_tokens</code> are already persisted
Gemini	Zero visibility today. Needs instrumenting at the call sites (<code>src/lib/scraper/query/generateQueries.ts</code> , <code>src/lib/scraper/city-expander.ts</code> , <code>src/app/api/leads/summary/route.ts</code> , <code>src/lib/whatsapp/*</code>) to write one sample per call. Defer if Phase 4 runs long.
Firecrawl / Apify / Google Places	<code>scraper_runs</code> counters (rolled up, not attributed per source)
Decentro / DigiO / CIBIL	<code>kyc_verifications</code> — one row per vendor API attempt, with <code>api_provider</code> and <code>status</code>
Razorpay / RazorpayX	<code>facilitation_payments</code> , <code>emi_payment_attempts</code> , <code>buyback_gateway_transactions</code>
Meta WhatsApp	<code>whatsapp_messages</code> — one row per message, inbound and outbound
Zoho	<code>zoho_sync_state</code> + <code>zoho_invoices.synced_at</code>
...	...

AWS S3	every object read flows through <code>src/app/api/files/[bucket]/[...path]/route.ts</code> — the single instrumentation point
--------	---

Billed spend (what we actually paid): `expense_submissions WHERE bucket='tech' GROUP BY vendor`. `src/lib/expenses/bucketRules.ts` already classifies invoices from Anthropic, OpenAI, AWS, Vercel, Supabase, Upstash, GitHub, Zoho, Razorpay, Decentro, DigiO, Bolna, ElevenLabs, Firecrawl, Apify, Hostinger, Cloudflare and more, and multi-currency is handled (`original_amount` , `fx_rate` — E-217). **No new integration is needed for this chart.**

Render: monthly burn, MoM delta, and a metered-vs-billed reconciliation table per vendor.

8.7 Business metrics — `/operations/business`

A thin read-only mirror of numbers the CEO dashboard already computes, so the tech team can see whether a deploy broke a pipeline. **Call the existing aggregations — do not re-derive them:** `src/app/api/dashboard/ceo/overview/route.ts` with `resolveWindowParams` from `src/lib/dashboard/salesWindow.ts` (every CEO route uses that resolver, which is why the numbers reconcile).

Metrics: leads created, leads converted, dealers onboarded, revenue, outstanding, buyback submitted/completed, active IoT devices. Snapshotted daily so a sudden drop to zero raises an alert.

8.8 Team usage — `/operations/team`

Nothing exists for this today. There is no `sessions` , `logins` , `last_seen` or `page_views` table, and `users` has no `last_login` column. Build it the cheap way:

- A collector calls the **Supabase Admin API** (`supabaseAdmin.auth.admin.listUsers()` , which returns `last_sign_in_at`) and writes one sample per user. Sessions are Supabase-managed and live in `auth.users` / `auth.sessions` , not in our Postgres.
- Actions-per-user from `audit_logs.performed_by` (36 API routes already write it), plus `ai_call_logs` , `dealer_leads.created_by` and `expense_submissions` .

Screen: active users 24 h / 7 d / 30 d, never-logged-in accounts, accounts inactive >30 days (an offboarding signal), actions by role, top actors.

Deliberately scoped to **aggregate** activity — this is capacity and licence monitoring, not per-person surveillance.

8.9 Alerts — `/operations/alerts`

Open alerts (severity, metric, value vs threshold, age) with acknowledge/resolve, plus an editable rules table backed by `ops_alert_rules` . Delivery: in-app via `notifyRoles(["operations"])` and email via `getMailer()` , gated by `cooldown_minutes` so a flapping metric cannot spam.

8.10 Collector health — `/operations/jobs`

`ops_collector_runs` as a table: collector, last run, duration, samples written, status, error. Manual "Run now" button per collector.

This answers "is the monitoring itself alive?" Without it, a silently-dead collector reads as "everything is green" — the exact failure mode this dashboard exists to prevent.

8.11 Backlog (after Phase 7, if wanted)

Daily email / WhatsApp digest · synthetic uptime checks with `puppeteer` (already installed; `scripts/perf-audit/` already does Lighthouse-style runs) · CloudWatch RDS metrics and Cost Explorer (needs a new IAM policy and SDK client) · RDS backup/snapshot age · weekly `npm audit` / outdated-dependency scan · S3 storage growth · domain-expiry tracking · per-source scraper attribution.

9. Wiring the role

`users.role` is a bare `varchar(50)` with **no enum and no CHECK constraint**, so this needs **no database migration**. But there is also **no single role registry** — a role is whatever six hardcoded lists agree it is, and **missing one fails silently** (the user falls back to the `user` nav, or gets redirected to `/` , with no error logged anywhere). See `src/lib/__tests__/roles.test.ts` , which documents exactly this trap.

The six lists

#	File	Change
1	<code>src/middleware.ts</code> — <code>roleDashboards</code> (~line 340)	add <code>operations: "/operations"</code> , . This alone makes <code>/operations/*</code> a protected route and sets the post-login landing page.
2	<code>src/components/layout/sidebar.tsx</code> — <code>roleNavigation</code> (~line 165)	add an <code>operations: [...]</code> block; copy the <code>it:</code> block at ~line 772
3	<code>src/components/layout/sidebar.tsx</code> — <code>inferredRole</code> (~line 1450)	<code>if (pathname.startsWith("/operations")) return "operations";</code>
4	<code>src/app/(auth)/login/page.tsx</code> (~line 168)	add the <code>operations</code> branch to the redirect if/else, next to the <code>it</code> branch
5	<code>src/app/(auth)/login/actions.ts</code> (~line 75)	add the same branch to the server-action login path
6	<code>src/lib/roles.ts</code> — <code>ROLE_PATH_PREFIXES</code>	add <code>["/operations", "operations"]</code> , before the <code>["/admin", "admin"]</code> tail

Also:

- add `"operations"` to `NO_COMMON_ITEMS` in `sidebar.tsx` (~line 1544) if this login should not get the "Submit Expense" item;
- add `/operations` to `showMobileDrawer` (`sidebar.tsx` ~line 1705) **and** to `showMobileNav` (`src/components/layout/header.tsx` ~line 29) — they are independent copies;
- add a case to `src/lib/__tests__/roles.test.ts` so the new role is pinned by a test.

Name check. The string `"operations"` is already used as an NBFC role, but that lives in `nbfc_users.role` and is resolved by `resolveActor(headers)` against the NBFC tenant — never against `users.role` . Verified: no code compares `users.role` to `"operations"` . There is no collision.

CEO bypass. `src/middleware.ts` lets the `ceo` role open every dashboard. Decide whether `ALLOWED_ROLES` on the ops pages is `{"operations"}` or

```
{"operations", "ceo"}.
```

API guard

`src/lib/operations/route-guard.ts` — copy `src/lib/security/route-guard.ts` verbatim and change the role set to `new Set(["operations"])`. Keep the **id-then-email** user lookup and the `is_active` check; both are load-bearing (Supabase lowercases emails while `users.email` is mixed-case — an email-keyed lookup is the classic "no session / wrong user" bug in this codebase).

First two lines of every `/api/operations/*` handler:

```
const auth = await requireOperationsAdmin();
if (!auth.ok) return auth.response;
```

The login

`scripts/create-operations-user.ts` — copy `scripts/create-it-user.ts`, set `ROLE = "operations"` and the default email to `operations@itarang.com`. Keep `import "../load-env";` as line 1 (`@/lib/db` throws if `DATABASE_URL` is unset). Register `"seed:operations-user"` in `package.json` next to `seed:it-user`.

The script must set **both**:

1. Supabase `app_metadata.role` — what `middleware.ts` reads;
2. the RDS `users` row — what every page and API guard reads.

Keyed on the Supabase auth UUID, **never on email**. Required columns: `id`, `email`, `name`, `role`, `is_active = true`, `must_change_password = false`, `password_hash`, `created_at`, `updated_at`.

Run `npm run seed:operations-user` once against database-1 (sandbox) and once against database-2 (prod).

Credentials: `operations@itarang.com` / `password`.

The password is fine for sandbox. Rotate it on prod once the dashboard is live — this login can read company spend and infrastructure detail.

10. File map

src/lib/operations/	
registry.ts	metric definitions (the add-a-metric file)
route-guard.ts	requireOperationsAdmin()
runner.ts	runDueCollectors(); begin/complete/fail run; single-flight lock
daily.ts	runDailySnapshot(); prune()
alerts.ts	evaluateAlerts(); open/resolve; cooldown; notify
format.ts	unit formatting shared by pages and email
collectors/	
types.ts	OpsCollector, CollectedSample
index.ts	export const COLLECTORS: OpsCollector[]
host.ts	derives metrics from ingested agent payloads
database.ts	pg_stat_* against CRM RDS + IoT bridge
appHealth.ts	/api/health + /api/health/redis
jobs.ts	*_runs tables + zoho_sync_state freshness
spend.ts	expense_submissions bucket='tech' by vendor
business.ts	CEO aggregations
team.ts	Supabase admin last_sign_in_at + audit_logs
vendors/	
elevenlabs.ts	subscription/credits + ai_call_logs rollup
bolna.ts	openai.ts razorpay.ts zoho.ts kyc.ts whatsapp.ts scraper.ts
src/app/api/operations/	
ingest/host/route.ts	POST – the agent's endpoint (bearer + allowlist + zod)
summary/route.ts	one-slide payload
metrics/[key]/route.ts	time series for a chart
infrastructure database logs system spend business team/route.ts	
alerts/route.ts	GET list; PATCH acknowledge/resolve
alerts/rules/route.ts	GET/PUT thresholds
jobs/route.ts	collector runs
jobs/run/route.ts	POST manual "run now"
export.xlsx/route.ts	ExcelJS report
src/app/(dashboard)/operations/	
page.tsx	one-slide
infrastructure database logs system spend business team alerts jobs/page.tsx	
_components/	OpsNavTabs, MetricTile, TrafficLightGrid, MetricSparkline, LogExplorer, AlertsTable, RulesEditor, RunCollectorButton, AutoRefresh (copy from it/security/_components)
drizzle/E-227_ops_monitoring.sql	
scripts/create-operations-user.ts	
ops-agent/agent.js	
ops-agent/README.md	
ops-agent/ecosystem.ops-agent.config.js	
docs/OPERATIONS_RUNBOOK.md	

11. Phases

Each phase is independently shippable and reviewable. Estimates assume one developer.

#	Phase	Scope	Done when	Est.
0	Foundation	The six role lists (§9); <code>route-guard.ts</code> ; <code>scripts/create-operations-user.ts</code> + npm script; empty <code>/operations</code> page with nav tabs	<code>operations@itarang.com</code> logs in and lands on <code>/operations</code> ; every other role is bounced	0.5 d
1	Metric spine	<code>E-227</code> + <code>schema.ts</code> mirror + checklist row; <code>registry.ts</code> ; collector types; <code>runner.ts</code> ; <code>startOpsMonitorTicker</code> ; <code>/operations/jobs</code>	One trivial collector (app uptime) runs on the ticker, writes samples, and its run row shows on <code>/operations/jobs</code>	1.5 d
2	Infrastructure + DB	<code>ops-agent/agent.js</code> ; ingest route + hardening; <code>host.ts</code> and <code>database.ts</code> collectors; <code>/operations/infrastructure</code> and <code>/operations/database</code>	Agent installed on prod + sandbox; both pages show live CPU/disk/pm2 and RDS connection headroom	2.5 d
3	Logs	Log tail + offset tracking in the agent; <code>ops_log_events</code> write path with fingerprinting; <code>/operations/logs</code> explorer + export	An error in a prod pm2 log appears within 5 minutes, grouped and filterable	1.5 d
4	Spend & vendor usage	Vendor collectors (ElevenLabs subscription first); <code>spend.ts</code> ; <code>/operations/spend</code> with metered-use billed reconciliation	ElevenLabs credit balance and 30-day spend per vendor render; the numbers tie out against <code>Readme - Cost Attribution</code>	2 d

		vs-billed reconciliation	/leads → lost Analytics	
5	Business + team	business.ts reusing the CEO aggregations; team.ts via the Supabase Admin API; both pages	Both pages match the CEO dashboard for the same window	1.5 d
6	One-slide, snapshots, alerts	runDailySnapshot; pruning; ops_alert_rules seeding; alerts.ts; /operations and /operations/alerts	Snapshot rows appear after the first 00:15 run; a forced threshold breach opens an alert, notifies exactly once, and auto-resolves on recovery	2 d
7	Hardening	Retention verified; vitest coverage; docs/OPERATIONS_RUNBOOK.md; prod rollout with secrets in both places	Type-check and tests clean on touched files; agent survives a box reboot	1 d

Total ≈ 12.5 developer-days.

12. Testing

vitest is configured — npm test. (The line in CLAUDE.md saying no test suite exists is stale.)

Unit (vitest, no DB):

- registry.ts — keys are unique; every key has a label and unit; warn sits on the correct side of crit for the metric's direction
- alerts.ts — a breach opens exactly one alert, stays open while breached, resolves on recovery, and respects cooldown_minutes
- runner.ts — due-calculation; timeout handling; a throwing collector fails its own run without killing the others
- ingest zod schema — oversized payload, skewed clock, unknown host and bad bearer are all rejected
- format.ts — paise → ₹, bytes → GB, percent rounding

Integration (against database-1 / sandbox):

- the migration applies twice cleanly (run it, run it again, diff nothing)
- single-flight lock: two concurrent runDueCollectors() calls produce one run row per collector
- runDailySnapshot() is re-runnable for the same date without duplicating rows

Manual:

- npm run seed:operations-user, log in, confirm the landing page, and confirm a dealer login is bounced away from /operations
- run the agent locally against http://localhost:3000 and watch a sample land
- lower a threshold to an always-breaching value and confirm the alert opens, notifies once, and resolves when the threshold is restored

13. Known landmines

Read these before touching the relevant area.

- /api/system/health is unauthenticated and dumps the DB host, port, user and password length. Gate it (or delete it) while working here — it undercuts every other access control on this dashboard.
- scraper_runs is declared twice in src/lib/db/schema.ts (scraperRuns ~line 2592 and scrapeRuns ~line 3193 — same table name, divergent column sets). Pick one when the jobs collector reads it.
- /api/system/database-monitor queries the IoT database, not RDS, despite its name and its requireRole(['ceo']) gate. Do not reuse it blind.
- Prod's shared/.env is overwritten from PROD_ENV_FILE_B64 on every deploy. New env vars need the box edit and the GitHub secret. Sandbox's is seeded once and edited on the box; changing the GitHub env secret does not propagate to sandbox.
- Migrations do not auto-run. An unapplied E-227 surfaces at runtime as relation "ops_metric_samples" does not exist — Drizzle names every column in its INSERTs.
- Never run npm run db:push against sandbox or production. It is diff-based and will drop columns.
- RDS has ~79 max_connections and no pooler. Collectors must reuse the existing db client (max: 5) and must never open their own pools.
- npm run type-check and npm run lint are already red on a clean tree. Filter output to your touched files to judge your own change.
- Several agent branches merge to main quickly. Re-check origin/main's current copy of middleware.ts, sidebar.tsx and instrumentation-node.ts before editing them — expect competing edits to the same lists.